

UNIT – 2 Version Control Systems

- PROF. PRACHI JAIN

UNIT - 2

- ▶ Centralized VCS and Distributed VCS
- ▶ Introduction to GIT
- ▶ SVN and Bitbucket
- ▶ Git Essentials
- ▶ Git Commandline
- ▶ Git architecture and versions of Git
- ▶ Cloning
- ▶ Check-in and Commit of Git Repositories
- ▶ Fetching the Repositories
- ▶ Git Pull and Git Branching Technique

Version Control Systems

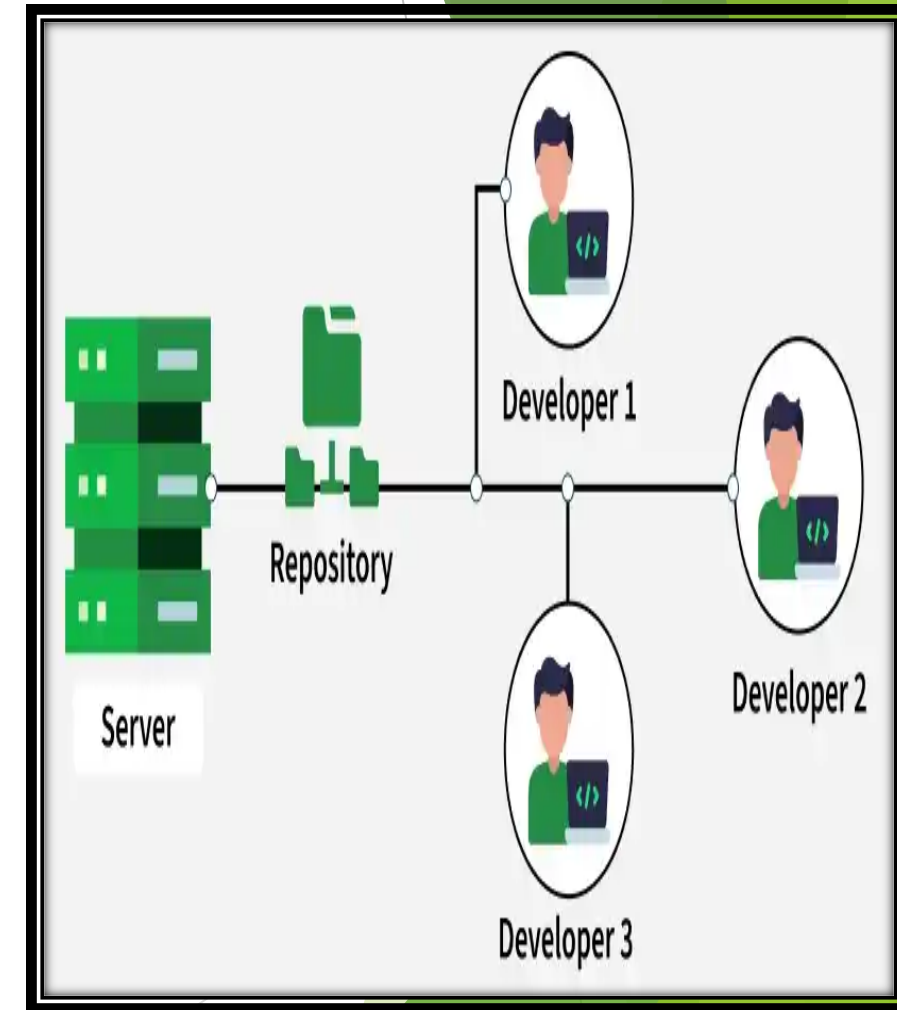
A Version Control System (VCS) is a tool used in software development and collaborative projects **to track and manage changes to source code.**

- ▶ Tracks and records changes to the codebase, maintaining a structured project history.
- ▶ Allows multiple developers to collaborate on the same project without overwriting each other's work.
- ▶ Developers access and update a central repository, where project files are stored.
- ▶ Enables teams to share updates and revert to earlier versions of the project when needed.

Working of the Figure:

1. The **Server** hosts the **central repository**.
2. All developers connect to this repository through the network.
3. Developers download the latest project files from the repository.
4. They modify the files on their local computers.
5. After completing their work, they **commit (check-in)** the changes back to the central repository.
6. Other developers can then **update** their local copies to receive the latest changes.

Component	Explanation
Server	The central server stores all project files and their complete version history. It acts as the main storage location.
Repository	The repository is the central database that contains the source code, documents, and all previous versions of the project.
Developer 1, Developer 2, Developer 3	Multiple developers connect to the central repository to download (checkout/update) files and upload (commit) their changes.



Components of Version Control Systems

Version Control Systems work using a few core concepts that help teams manage code changes and collaborate efficiently.

- ▶ **Repository:** A central location that stores all project files along with their complete change history and metadata like author and commit message.
- ▶ **Revision:** A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- ▶ **Branch:** A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- ▶ **Merging:** The process of combining changes from one branch into another, which may sometimes require resolving conflicts.
- ▶ **Commit:** A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

Types of Version Control Systems

Local Version Control Systems (Local VCS)

Centralized Version Control Systems

Distributed Version Control Systems

Types of Version Control Systems

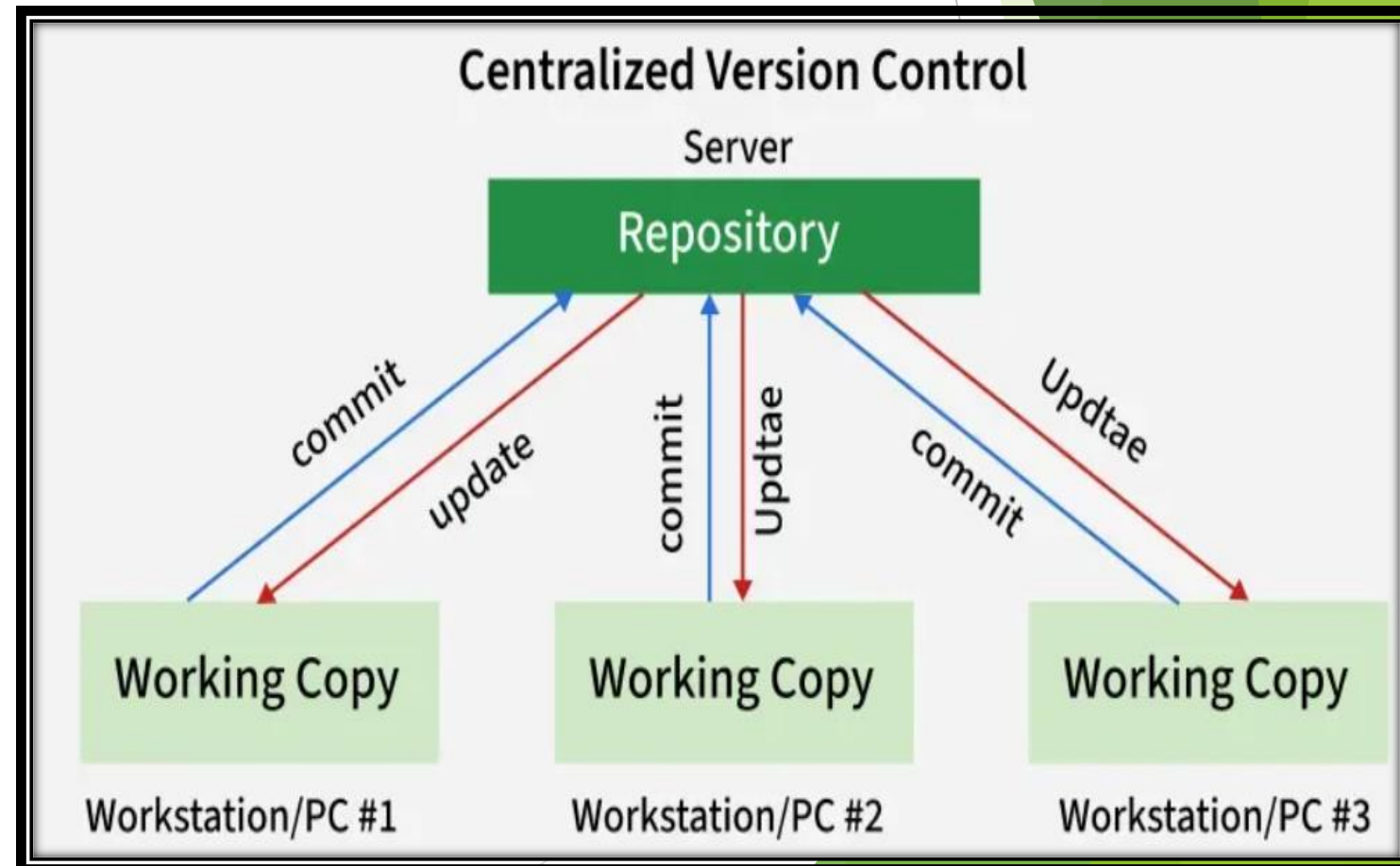
1. Local Version Control Systems (Local VCS)

A Local Version Control System (LVCS) stores all project versions on a single computer and is mainly used by one user without remote collaboration.

- ▶ **Characteristics:**
- ▶ No internet or server dependency.
- ▶ Useful for individual projects.
- ▶ Limited to single-user environments

2. Centralized Version Control Systems

- ▶ In a Centralized Version Control System, all the files and their version history are stored in a single central server.
- ▶ Developers connect to this server to access or modify files.
- **Update/Checkout:** A developer retrieves the latest version of the files from the central server.
- **Make Changes:** The developer works on the files.
- **Commit:** The developer saves (commits) the changes directly back to the central server, making them immediately available to everyone else.



Pros:

- ▶ Enables collaboration among multiple developers through a central repository.
- ▶ Provides visibility into project activities and changes.
- ▶ Allows fine-grained access control for administrators.

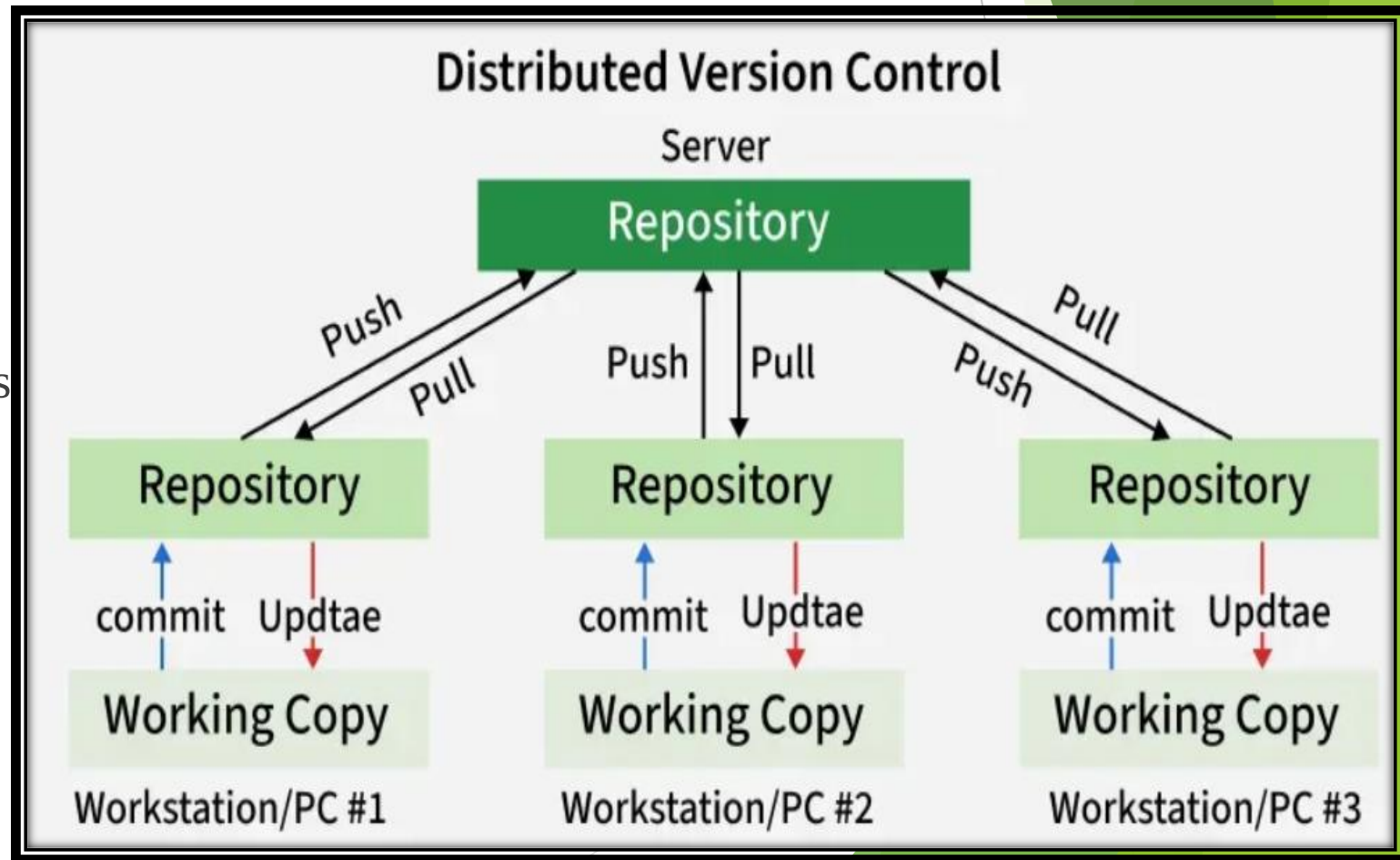
Cons:

- ▶ Has a single point of failure because everything depends on the central repository.
- ▶ If the server goes down, developers cannot collaborate or commit changes.

3. Distributed Version Control Systems

- ▶ A Distributed Version Control System allows each developer to have a local repository along with a working copy of the project.
- ▶ Changes made in the local repository are not automatically visible to others.

- ▶ **Commit:** Saves changes to the local repository, visible only to the developer.
- ▶ **Push:** Uploads committed changes to the central/shared repository so others can access them.
- ▶ **Pull:** Downloads changes from the central repository to the local repository.

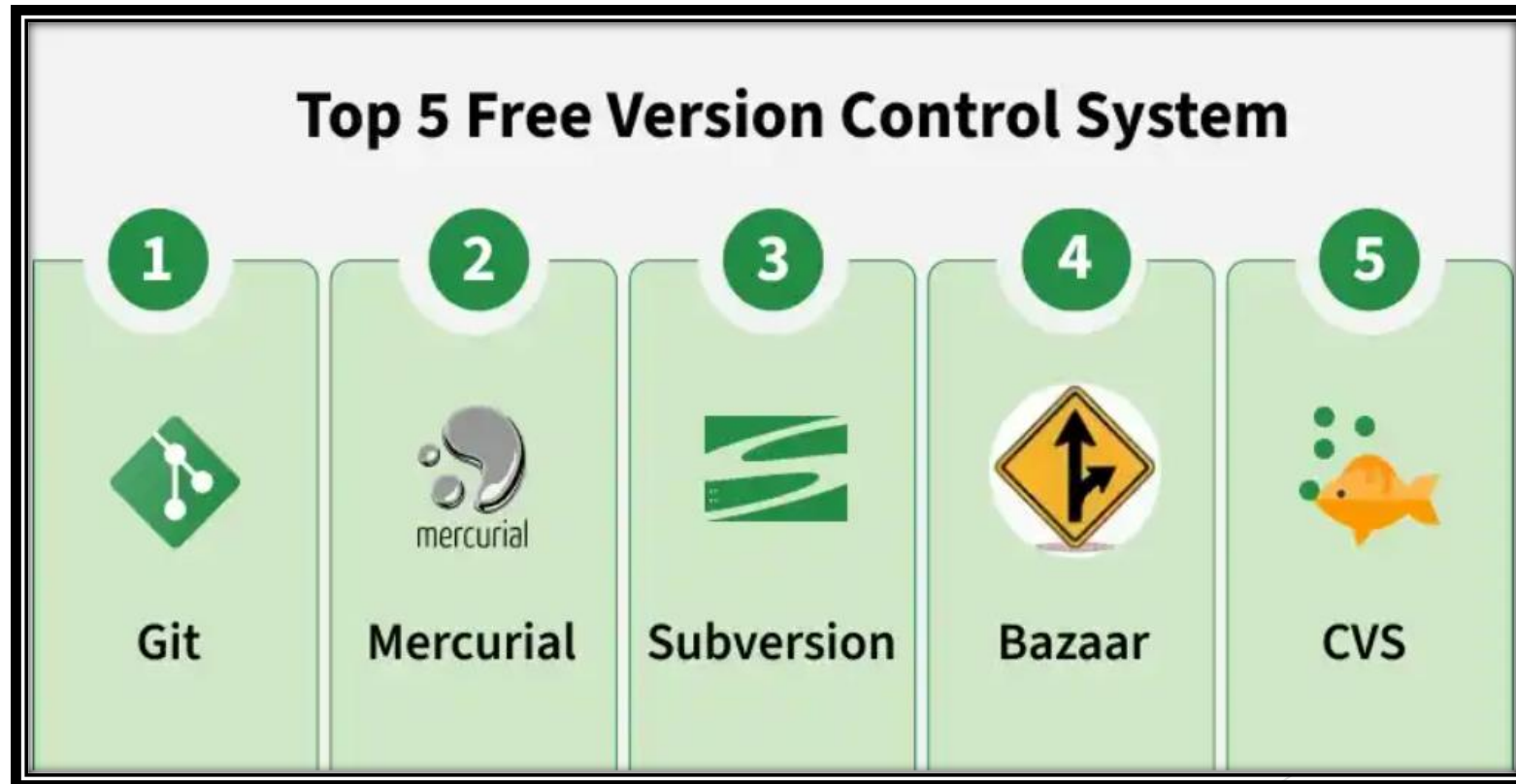


Difference between CVCS and DVCS

Feature	Centralized VCS	Distributed VCS
Repository	One central repository	Every user has full repository
Internet Requirement	Required for most operations	Not required for local work
Speed	Slower	Faster
Backup	Only central server	Every clone is backup
Failure	Server failure stops work	Users continue working
Commit	Directly to server	Local first, then remote
Collaboration	Moderate	Excellent
Branching	Slow	Very fast
Examples	Subversion and Concurrent VCS	Git, Mercurial

Popular Version Control Systems

- ▶ Below are five most widely used free VCS tools, perfect for individuals and teams.
- ▶ Each of these version control systems serves different needs, and the choice depends on the project size, team collaboration style and workflow preferences.



Version Control System	Type	Developed	Description	Key Features
Git	Distributed Version Control System (DVCS)	Linus Torvalds (2005)	- A widely used distributed version control system developed for managing the Linux kernel. - It supports decentralized workflows and is used by platforms like GitHub, GitLab, and Bitbucket.	- Lightweight, fast, and efficient. - Simple and non-destructive branching and merging. - Provides powerful commands such as git clone, git pull, and git push.
Subversion (SVN)	Centralized Version Control System (CVCS)	Apache Software Foundation	A popular centralized version control system widely used in organizations and enterprises for its simplicity and centralized repository structure.	- Single central repository. - Supports branching and tagging, but less flexible than Git. - Provides versioning of files and directories.
Mercurial	Distributed Version Control System (DVCS)	Matt Mackall (2005)	- A distributed version control system similar to Git with a simpler interface. - Suitable for both small and large projects.	- Simple, fast, and scalable. - Supports branching and merging. - Includes tools for managing project history and tracking changes.

Version Control System	Type	Developed	Description	Key Features
CVS (Concurrent Versions System)	Centralized Version Control System (CVCS)	Dick Grune (1986)	• One of the earliest centralized version control systems. • It influenced modern VCS tools such as Subversion.	• Centralized repository architecture. • Tracks changes to individual files over time. • Supports multiple developers working simultaneously. • Basic support for branching and tagging.
Bazaar	Distributed Version Control System (DVCS)	Canonical Ltd.	• A distributed version control system that supports both centralized and distributed workflows. • It was used in projects such as Ubuntu and Launchpad.	• Supports both centralized and distributed workflows. • Beginner-friendly and easy to learn. • Uses human-readable commands such as <code>bzr commit</code> and <code>bzr push</code>. • Cross-platform support (Linux, Windows, and macOS).

Git Introduction

- ▶ Git is a distributed version control system (VCS) used to track changes in source code during software development.
- ▶ It helps developers collaborate, manage different versions of code, and roll back to previous states if needed.
- Multiple developers can work together and merge changes easily.
- Revert to previous versions whenever needed.
- Develop features separately and merge them safely.
- Each developer has a full copy of the repository.
- Stores snapshots of the entire tree and optimizes storage internally using compression and delta encoding.

GIT INTRODUCTION

- Git is an **open-source distributed version control system** developed by **Linus Torvalds** in **2005**.
- It helps developers:
 - Track changes
 - Collaborate efficiently
 - Manage multiple versions
 - Merge code safely

► Features

- Distributed architecture
- Fast performance
- Secure
- Easy branching
- Supports offline work

► Advantages

- High speed
- Easy collaboration
- Reliable
- Excellent branching
- Free and open-source

- ▶ Git is a distributed version control system that helps track and manage changes in code over time.
- ▶ It allows multiple developers to work on the same project, view previous versions, and restore changes when needed.

- ▶ **Features of Git**

1. Version Control System

2. Repositories

- ❑ Local Repository
- ❑ Remote Repository

3. Commits

4. Branches

5. Merging

6. Cloning

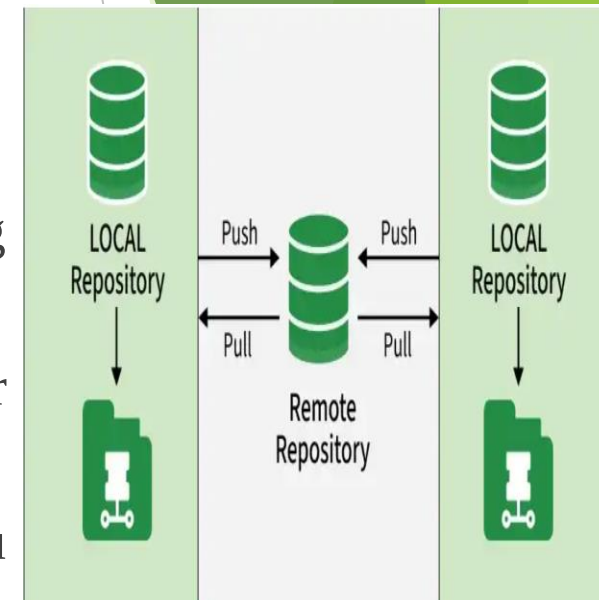
Features of Git

1. Version Control System:

- ▶ Git keeps track of every change you make to your project files.
- ▶ You can go back to previous versions if any requirement arises.

2. Repositories:

- ▶ A Git repository (or repo) is like a project's central hub where everything related to your work is stored and managed. There are two main types:
 - **Local Repository:** This is a copy of the repository which is stored in your computer. You can work on your project and make changes here.
 - **Remote Repository:** This is stored on a server, like GitHub, where you and others can share and work on a project.

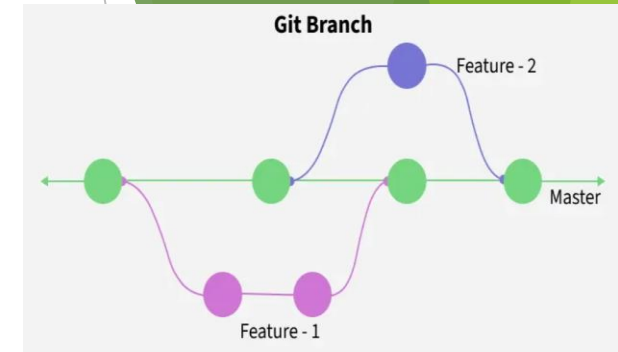


3. Commits:

- ▶ Every time you make changes and save them in Git, these are called commits.
- ▶ The repository keeps track of all the commits you have made.

4. Branches:

- ▶ A repository allows you to create Branches and work on new features and fixes.
- ▶ Common branch types include:
 - ▶ **Main (or Master) Branch:** The stable version of the project, usually production-ready.
 - ▶ **Feature Branch:** Used for developing new features or bug fixes.
- ▶ For example, you can have a "main" branch for stable code and a "feature" branch for new features you're developing.



5. Merging:

- ▶ Once you are done with the branches you have made, you can merge those branches into main branch.
- ▶ This adds the changes you have made in the rest of the project.

6. Cloning:

- ▶ Cloning is a process of making a complete copy of Git repository.
- ▶ It's like copying the entire project from central location (like website) to your own computer.

► The Three States (The Staging Area)

- In Git, your files can be in one of three main states, understanding this is key to understanding how Git works:
- **Working Directory:** This is your project folder with all the files you are currently working on. Any changes you make here are not yet tracked by Git.
- **Staging Area (or Index):** This is an intermediate area where you list the specific changes you want to include in your next "snapshot" or commit. You use the `git add` command to move changes from your working directory to the staging area.
- **Repository (.git directory):** This is where Git permanently stores the snapshots (commits) of your project. You use the `git commit` command to save the staged changes into the repository.
- This three-step process (modify -> add -> commit) gives you precise control over what gets saved in your project's history.

Feature	Git	GitHub
Definition	Git is a tool used for version control to track code changes.	GitHub is a platform that hosts Git repositories online.
Developer	Created by Linus Torvalds .	Developed by Tom Preston-Werner, Chris Wanstrath, and PJ Hyett .
Company	Managed by the Linux Foundation .	Owned by Microsoft .
Release Year	Released in 2005 .	Launched in 2008 .
Interface	Git is mainly a command-line tool.	GitHub provides a graphical user interface (GUI) .
Installation	Git is installed locally using the command git init .	GitHub is a cloud-based service, accessible via a browser.
Version Control	Git is a distributed version control system (DVCS) that tracks code changes.	GitHub provides hosting for Git repositories with additional collaboration features.
User Management	Git does not have built-in user management.	GitHub allows team collaboration and access control.
Cost & Availability	Git is free and open-source .	GitHub has both free and paid plans.
Collaboration	Git is mainly for individual developers working locally.	GitHub helps teams collaborate with features like pull requests and issues .
Integration & Extensions	Git works independently with minimal external tools.	GitHub supports third-party integrations and marketplace extensions .
Alternatives & Competitors	Git competes with Apache Subversion (SVN), Mercurial and Perforce .	GitHub competes with GitLab, Bitbucket, AWS CodeCommit and Azure DevOps .

SVN (Subversion)

- SVN is a **Centralized Version Control System**.
- Developers store and retrieve files from one central server.

Features

- Central repository
- Version tracking
- Access control
- Supports binary files

Advantages

- Easy to understand
- Central administration
- Good permission management

Disadvantages

- Requires network
- Single point of failure
- Slow branching

GIT	SVN
Distributed version control system by Linus Torvalds (2005)	Centralized version control system under Apache license
Git has a Distributed Model.	SVN has a Centralized Model.
Each user has a full local copy of the repository	Single central repository with working copies
Works offline without network	Requires network for most operations
More complex with many commands	Easier to learn and use
Handles large and binary files less efficiently	Better support for binary files
Uses a single .git directory	Uses .svn directory
Limited UI support	Simpler and better UI

Bitbucket

Bitbucket is a Git-based repository hosting service used for version control, code management, and team collaboration.

- Provides Git-based version control for managing source code.
- Offers integration with tools like Jira and CI/CD pipelines.
- Allows both private and public repositories.
- Enables efficient team-based development workflows.

Features

Provides tools for efficient code management, collaboration, and project tracking.

- **Code Review System:** Fast pull request reviews for developers and reviewers.
- **JIRA Integration:** Easily integrates with JIRA for bug tracking.
- **In-line Discussion:** Supports comments and threaded conversations in code.
- **Cloud & Server Options:** Provides cloud for small teams and server for large teams.
- **Built-In Issue Tracker:** Tracks issues with configurable fields like version and milestone.
- **User-Friendly Interface:** Simple interface for easy navigation and search.

Advantages

Offers integrated tools, easy authentication, and cost-effective repository management.

- ▶ **Query Management System:** Simplifies work with an integrated query management system.
- ▶ **Authentication via GitHub:** Allows authentication and linking through GitHub accounts.
- ▶ **Integrated JIRA Tool:** Supports bug tracking with automatic updates on commits.
- ▶ **Import Repositories:** Enables importing repositories from third-party services.
- ▶ **Pricing:** Free for small teams with unlimited private repositories and academic benefits.

Disadvantages

Includes certain limitations in features, scalability, and overall user experience.

- ▶ Limited PR management features compared to competitors.
- ▶ Free private repositories restricted to teams of up to 5 members.
- ▶ Integrated tools like SourceTree may cause unexpected errors.
- ▶ Does not track details of force push, leading to loss of history.
- ▶ Search functionality can be slow and less efficient.

Bitbucket

- Bitbucket is a **Git repository hosting service** developed by **Atlassian**.
- It provides cloud-based Git repository management.
- **Features**
 - Git repository hosting
 - Pull requests
 - Code review
 - CI/CD integration
 - Issue tracking
 - Access management
- **Advantages**
 - Team collaboration
 - Secure repositories
 - Easy integration with Jira
 - Cloud backup

Feature	Git	SVN	Bitbucket
Type	DVCS	CVCS	Git Hosting Platform
Repository	Local + Remote	Central Server	Remote Git Repositories
Offline Support	Yes	Limited	Depends on Git
Branching	Fast	Slow	Uses Git Branching
Developed By	Linus Torvalds	Apache	Atlassian

Git Essentials

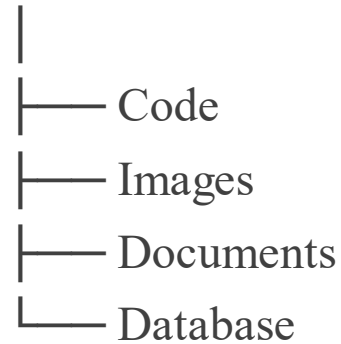
- ▶ Git Essentials are the **basic concepts** that every developer should understand before using Git.
- ▶ These concepts help us:
 - ▶ Track changes in project files
 - ▶ Manage different versions of code
 - ▶ Collaborate with team members
 - ▶ Restore previous versions when needed
- ▶ **Remember:** Git records every important change made to a project.

[A]**Repository (Repo)** is a folder managed by Git that stores:

- Project files
- Folder structure
- Complete version history

► **Example**

► CollegeProject/



► **Key Points**

- ✓ Git tracks every file inside the repository.
- ✓ Every modification is stored as a version.
- ✓ Repository acts like a digital storage for your project.

► **Real-Life Example:**

Think of a repository as a **school bag** that safely stores all your books and notebooks.

[B] Working Directory

- ▶ The **Working Directory** is the place where developers create, edit, and delete project files.
- ▶ **Example**
 - ▶ index.html
 - ▶ style.css
 - ▶ app.js
- ▶ These files are edited in the Working Directory.

[C] Staging Area (Index)

- ▶ The **Staging Area** is a temporary place where selected files are kept before saving them permanently.
- ▶ **Real-Life Example**
 - Student writes Assignment [Working Directory]
 - ↓
 - Teacher checks Assignment [Staging Area]
 - ↓
 - Final Submission
- ▶ Git follows the same process.

Git Workflow

Working Directory



Staging Area



Repository

Key Point:

The Staging Area lets you choose which changes should be included in the next commit.

[D] Commit

- ▶ A **Commit** is a permanent snapshot of the project at a particular point in time.
- ▶ Each commit saves:
 - Code changes
 - Date and time
 - Author information
 - Commit message

▶ **Example**

Commit 1

Added Login Page



Commit 2

Added Registration

▶ **Real-Life Example:**

A commit is like pressing the **Save** button after completing your work.

Key Points

- ✓ Every commit creates a new version.
- ✓ Previous versions remain safe.
- ✓ Developers can return to any previous version.

[E] HEAD in GIT

- ▶ **HEAD** is a pointer that always points to the **latest commit** in the current branch.

- ▶ **Example**

Commit 1



Commit 2



Commit 3 ← HEAD

- ▶ **HEAD is Important:**

- Shows the current working version.
- Helps Git know where new commits should be added.
- Moves automatically after every new commit.

- ▶ **Example**

After creating another commit:

Commit 1



Commit 2



Commit 3



Commit 4 ← HEAD

Complete Git Workflow

Create/Edit File



Working Directory



`git add`



Staging Area



`git commit`



Repository

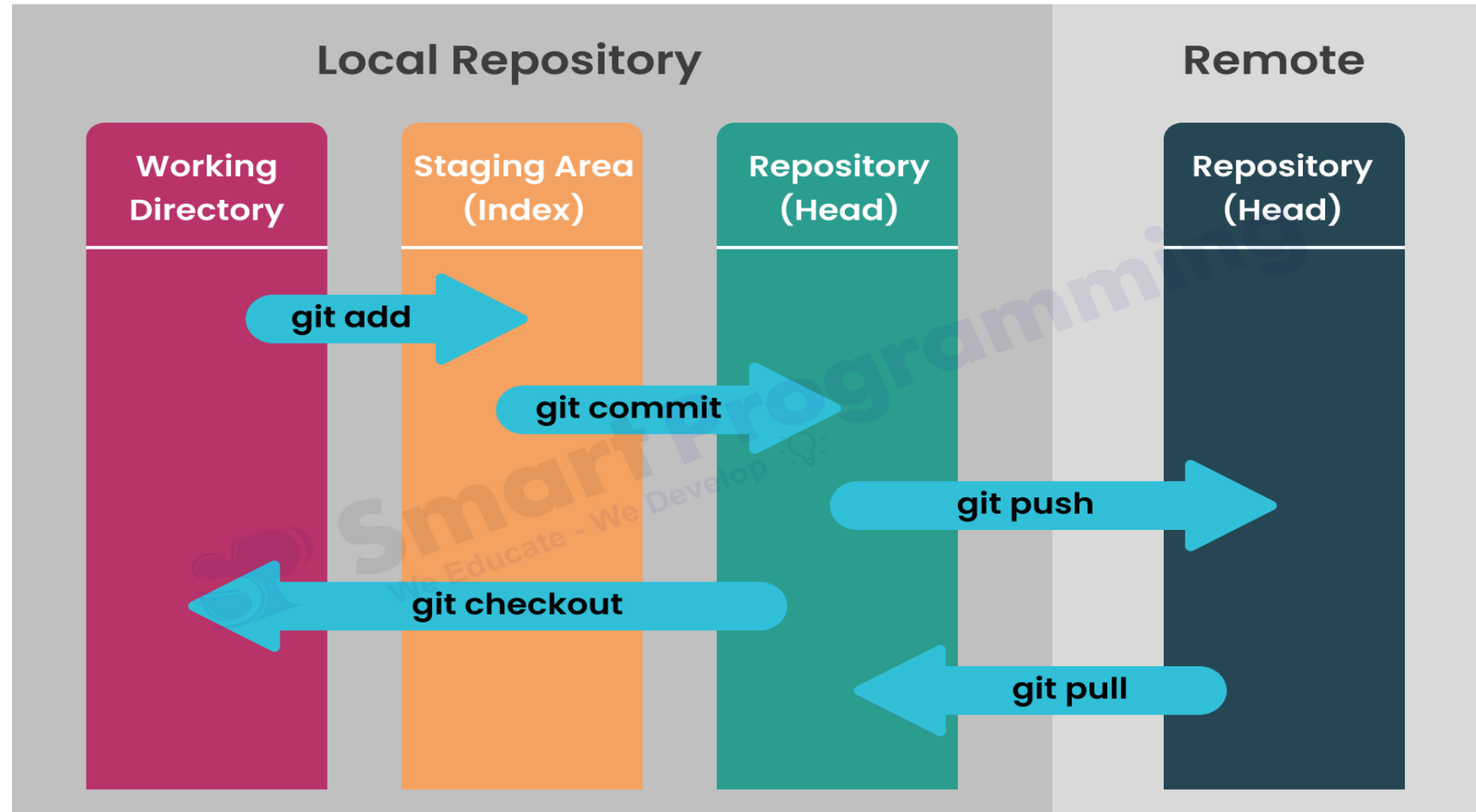


HEAD points to Latest Commit

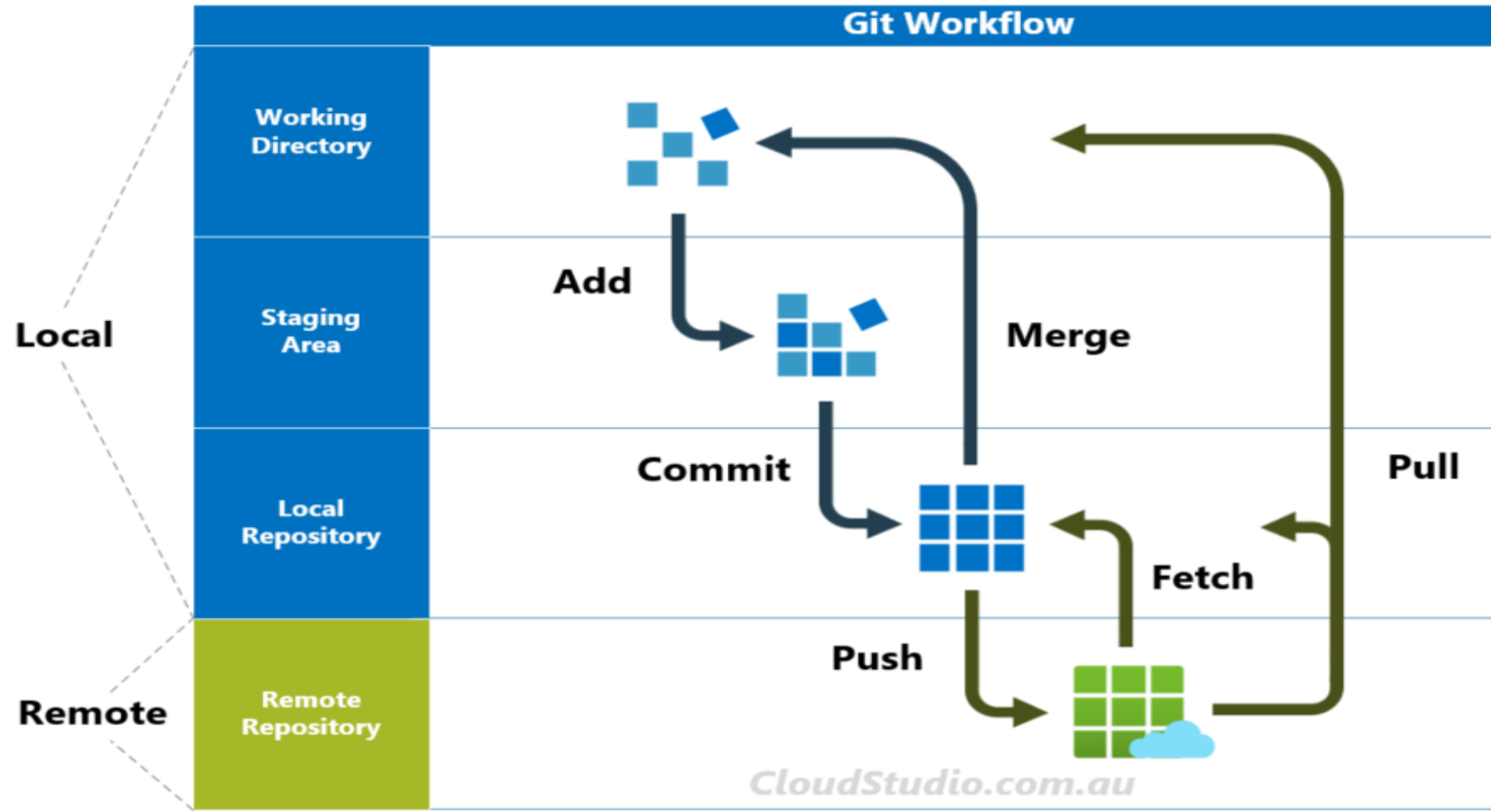
Git Essentials provide the foundation for version control, making it easier to manage projects, track changes, and collaborate with others.

Git Concept	Description
Repository	Stores project files and version history
Working Directory	Area where files are created and edited
Staging Area	Temporary area before committing
Commit	Permanent snapshot of project changes
HEAD	Points to the latest commit

Git Workflow & Stages [GIT Architecture]



Git Workflow



1. Working Directory:

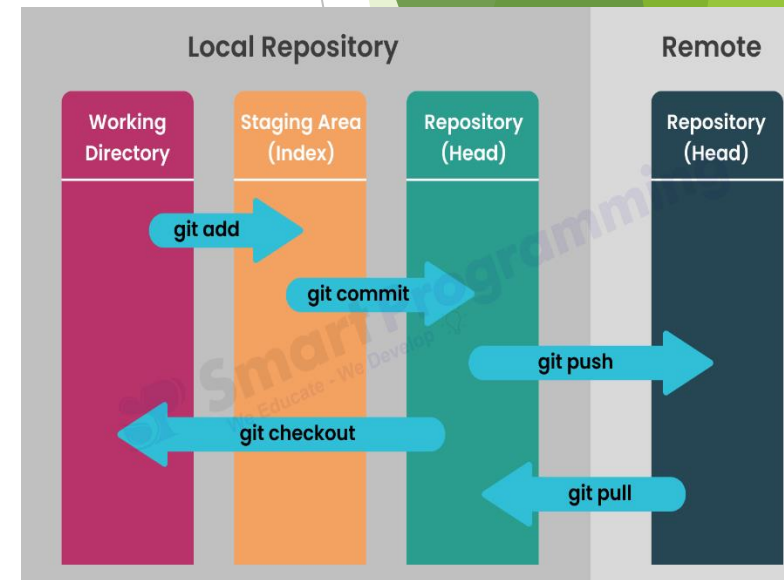
- This is where you can **create, edit, delete and modify** files in your project.
- Changes in this area are **not tracked** by Git until explicitly added.
- Command Used:
 - `git status` → Check which files are modified.
 - `git add <file>` → Move files from the working directory to the staging area.

2. Staging Area (Index):

- It is also known as the **Index**, this is where Git **temporarily stores changes** before committing them.
- It allows you to **prepare specific changes** before committing.
- Command Used:
 - `git commit -m "message"` → Moves files from the staging area to the local repository.

3. Local Repository (Head):

- Here, all committed changes are stored in **Git's history**.
- This is your **local copy** of the repository.
- Command Used:
 - `git push origin <branch>` → Pushes committed changes to the remote repository.

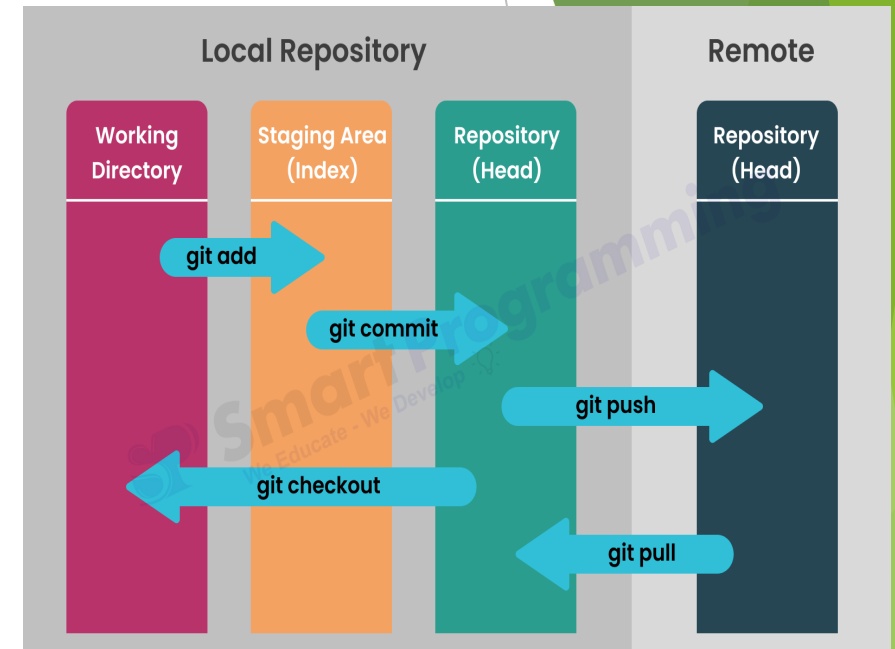


1.Remote Repository:

- It is a central repository (e.g. **GitHub, GitLab, Bitbucket**) where the project is stored online.
- Team members can **collaborate by pulling and pushing changes**.
- Command Used:
 - git pull origin <branch>** → Fetches and merges the latest changes from the remote repository.

2.Additional Commands:

- Some additional commands you can use are as follows :
 - git checkout** → It moves changes from the repository back to the working directory.
 - git pull** → It fetches and updates your local repository with changes from the remote repository.



Git Commandline

Git command line is an interface where you type and execute Git commands

- ▶ Git Bash
- ▶ Command Prompt
- ▶ Terminal
- ▶ PowerShell

Features of the Git Command Line

- ▶ The **Git Command Line** (Git CLI) is a text-based interface used to interact with Git.
- ▶ It enables developers to manage repositories, track changes, collaborate with teams, and automate software development tasks efficiently.

Feature	Description
1. Distributed Version Control	Every developer has a complete copy of the repository, allowing work even without an internet connection.
2. Fast and Efficient	Git command-line operations such as committing, branching, and merging are fast and consume minimal system resources.
3. Branching and Merging	Easily create, switch, merge, and delete branches for parallel development without affecting the main project.
4. Change Tracking	Records every modification made to files, making it easy to review and restore previous versions.
5. Command-Line Automation	Supports scripting and automation of repetitive tasks using shell scripts and batch files.
6. Collaboration Support	Allows multiple developers to work on the same project using remote repositories such as GitHub, GitLab, or Bitbucket.

Feature	Description
7. Repository Management	Enables users to create, clone, initialize, and manage local and remote repositories.
8. Powerful History Management	Commands like git log, git diff, and git blame help analyze project history and identify code changes.
9. Secure Data Integrity	Git uses SHA hashing to uniquely identify commits and ensure data integrity.
10. Cross-Platform Support	Git Command Line is available on Windows, Linux, and macOS.
11. Open Source	Git is free, open-source software that is widely used in industry and supported by a large community.
12. Lightweight	Git requires minimal storage and memory, making it suitable for projects of all sizes.

Advantages of Git Command Line

- ▶ Faster than many graphical Git tools.
- ▶ Gives complete control over Git operations.
- ▶ Suitable for automation using scripts.
- ▶ Supports advanced Git features.
- ▶ Works efficiently with large projects.
- ▶ Preferred by professional developers and DevOps engineers.

Applications of Git Command Line

- ▶ Source code version control.
- ▶ Team collaboration.
- ▶ Software development and maintenance.
- ▶ Continuous Integration and Continuous Deployment (CI/CD).
- ▶ Open-source project management.
- ▶ Backup and recovery of project versions.

Create

- ▶ Used to start working with repositories.
- ▶ **git clone:** Clones (copies) an existing remote repository to your local system, including all files, branches, and commit history.
- ▶ **git init:** Initializes a new Git repository in your current folder by creating a .git directory to track changes.

Local Changes

Used to track, review, and prepare changes before committing.

- ▶ **git status:** Displays the state of your working directory (modified, staged, and untracked files).
- ▶ **git diff:** Shows differences between working directory and staging area (what changes are not yet staged).
- ▶ **git add:** Adds all current changes to the staging area (prepares them for commit).
- ▶ **git add -p:** Allows you to stage changes selectively (line-by-line or chunk-by-chunk).
- ▶ **git commit -a:** Commits all tracked files directly without using git add (does not include new/untracked files).
- ▶ **git commit:** Saves staged changes as a new commit in the repository.
- ▶ **git commit --amend:** Modifies the last commit (useful for fixing commit message or adding missed changes).

Commit History

Used to view and analyse project history.

- ▶ **git log:** Shows all commits in reverse chronological order (latest first).
- ▶ **git log -p:** Displays detailed changes (diff) introduced in each commit.
- ▶ **git blame:** Shows who last modified each line of a file and when (useful for debugging).

Branches and Tags

Used for parallel development and version management.

- ▶ **git branch -av:** Lists all local and remote branches.
- ▶ **git checkout:** Switches from one branch to another.
- ▶ **git branch:** Creates a new branch from the current HEAD.
- ▶ **git checkout --track:** Creates a new branch that tracks a remote branch.
- ▶ **git branch -d:** Deletes a local branch (only if merged).
- ▶ **git tag:** Marks a specific commit (commonly used for releases like v1.0).

Update and Publish

Used to interact with remote repositories.

- ▶ **git remote -v:** Lists all configured remote repositories with URLs.
- ▶ **git remote show:** Displays detailed information about a specific remote.
- ▶ **git remote add:** Adds a new remote repository.
- ▶ **git fetch:** Downloads changes from remote repository without merging them.
- ▶ **git pull:** Fetches changes and merges them into your current branch.
- ▶ **git push:** Uploads your local commits to a remote repository.
- ▶ **git branch -dr:** Deletes a remote-tracking branch.
- ▶ **git push --tags:** Pushes all tags to the remote repository.

Merge and Reuse

Used to combine work from multiple branches.

- ▶ **git merge:** Merges another branch into your current branch, preserving history.
- ▶ **git rebase:** Reapplies your commits on top of another branch to maintain a clean linear history.
- ▶ **git rebase --abort:** Cancels the rebase process and returns to original state.
- ▶ **git rebase --continue:** Continues rebase after resolving conflicts.
- ▶ **git mergetool:** Opens a graphical tool to resolve merge conflicts.
- ▶ **git add / git rm:** After resolving conflicts manually, these commands mark files as resolved.

Undo

- ▶ Used to undo or revert changes safely or forcefully.
- ▶ **git reset --hard HEAD:** Discards all local changes and resets to the last commit (irreversible).
- ▶ **git checkout HEAD <file>:** Restores a specific file to its last committed version.
- ▶ **git revert:** Creates a new commit that undoes a previous commit (safe for shared repositories).
- ▶ **git reset --hard <commit>:** Moves HEAD to a previous commit and deletes all changes after it.
- ▶ **git reset:** Unstages changes but keeps them in the working directory.
- ▶ **git reset --keep:** Resets commits while preserving uncommitted local changes.

1. git init

- Initializes a new Git repository in the current directory.

git init

2. git clone

- Creates a copy of an existing remote repository.

git clone <repository-url>

3. git status

- Displays the state of the working directory and the staging area.
- It shows which changes have been staged, which haven't, and which files are not being tracked by Git.

git status

4. git add

- Adds changes from the working directory to the staging area.

git add <file>

To add all changes:

git add

*General
important GIT
Commands*

5. **git commit**

- Records the changes in the staging area in the repository with a descriptive message.

`git commit -m "Your commit message"`

6. **git log**

- Shows the commit history for the current branch.

`git log`

7. **git branch**

- Lists all branches in the repository. The * indicates the current branch.

`git branch`

Creates a new branch:

`git branch <branch-name>`

8. **git checkout**

- Switches to a different branch or restores files in the working directory.

`git checkout <branch-name>`

9. git merge

- Merges changes from one branch into the current branch.

git merge <branch-name>

10. git pull

- Fetches changes from a remote repository and merges them into the current branch.

git pull origin <branch-name>

11. git push

- Uploads local commits to a remote repository.

git push origin <branch-name>

12. git remote

- Manages the set of tracked repositories.

To add a new remote:

git remote add <name> <url>

13. git fetch

- Downloads objects and refs from another repository.

git fetch

14. git reset

- Resets the current HEAD to a specified state. It can be used to unstage changes or move the branch pointer.

To unstage changes:

git reset <file>

15. git diff

- Shows changes between commits, commit and working tree, etc.
- To see changes in the working directory:

git diff

- To see changes between the working directory and the index:

git diff --cached

16. git stash

- Temporarily shelves changes in the working directory that are not ready to be committed.

git stash.

17. git tag

- Creates a tag to mark a specific point in the repository's history, typically used for releases.

```
git tag <tag-name>
```

18. git blame

- Shows who made changes to each line in a file.

```
git blame <file>
```

19. git reflog

- Records updates to the tip of branches. It's useful for recovering lost commits.

```
git reflog
```

20. git rm

- Removes files from the working directory and the index.

```
git rm <file>
```

VERSIONS OF GIT

1. Torvalds (2005)
2. Usability (2.0)
3. Switch/Restore (2.23)
4. Scale/Sparse Checkout
5. SHA-256 (3.0)

1] Origin & Git 1.0 (2005) — The Foundation

- **Created By:** Linus Torvalds in **2005** to manage the Linux Kernel development.
- **Core Philosophy:** Designed to be highly **distributed** (every developer has a full copy of the project history) and incredibly fast.
- **Initial Focus:** Speed, data integrity (using SHA-1 encryption), and support for non-linear development (rapid branching).

2. Git 2.0 (2014) — The Usability Leap

- **User-Friendly Defaults:** Changed default behaviors to make Git safer for beginners.
- For example, `git push` was updated to only push the *current* active branch instead of all branches.
- **Refined CLI:** Cleaned up command-line arguments to make the tool intuitive for mainstream software teams.

3. Git 2.23 (2019) — Command Simplification

- **The Problem:** The old git checkout command was overloaded because it was used for both switching branches *and* restoring deleted files.
- **The Solution:** Git introduced two dedicated, easy-to-understand commands:
 - git switch: Used solely for changing branches.
 - git restore: Used solely for discarding local changes and restoring files.

4. Modern Git 2.x — Performance & Monorepos

- **Sparse Checkout:** Introduced the ability to checkout only a specific directory of a massive codebase instead of downloading gigabytes of files you don't need.
- **Performance Tuning:** Added partial clones and commit-graph optimizations to handle giant, enterprise-level repositories (like those at Google or Microsoft) seamlessly.

5. Git 3.0 (2026/Upcoming) — Security & Speed

- **SHA-256 Support:** Migrating the core cryptographic hash from the aging SHA-1 to SHA-256 to guarantee long-term security.
- **Reftable Backend:** A new system that speeds up reference lookup (branches/tags) by up to 20x in repositories with hundreds of thousands of active branches.

Git Versions

- ▶ Since its initial release, Git has been continuously updated with new versions that improve performance, security, usability, and compatibility.

Version	Release Year	Key Features
Git 1.0	2005	First stable release with basic version control features.
Git 1.5	2007	Improved branching, merging, and repository management.
Git 2.0	2014	Major update with performance improvements and new default behaviors.
Git 2.20	2018	Better support for large repositories, improved security, and bug fixes.
Git 2.30	2021	Faster operations, enhanced merge strategies, and improved maintenance.
Git 2.40	2023	Performance enhancements, security updates, and better command-line experience.
Latest Versions (2.x series)	Ongoing	Regular updates with bug fixes, new features, performance optimization, and security improvements.

Evolution of Git

Period	Development
2005	Git was created by Linus Torvalds for Linux kernel development.
2007–2013	Branching, merging, and collaboration features improved.
2014	Git 2.0 introduced major performance and usability improvements.
2015–Present	Continuous updates focused on speed, security, cloud integration, and better developer experience.

Features Added in Newer Git Versions

- ▶ Improved performance.
- ▶ Better branch and merge management.
- ▶ Enhanced security.
- ▶ Faster cloning and fetching.
- ▶ Support for large repositories.
- ▶ Improved conflict resolution.
- ▶ Better compatibility with hosting platforms like GitHub and GitLab.
- ▶ Bug fixes and stability improvements.

CLONING IN GIT

- ▶ **Git Cloning** is the process of creating a complete copy of an existing Git repository from a remote server (such as GitHub, GitLab, or Bitbucket) to a local computer.
- ▶ The cloned repository contains all the project files, commit history, branches, and configuration, allowing developers to work on the project locally.
- ▶ **Git Clone** is a Git command used to download an existing repository, including all its files, branches, and commit history, from a remote repository to a local machine.

► Syntax

```
git clone <repository_url>
```

► Example

```
git clone https://github.com/username/project.git
```

How Git Cloning Works

- ▶ A developer obtains the URL of the remote repository.
- ▶ The `git clone` command is executed.
- ▶ Git creates a new folder on the local machine.
- ▶ All project files, branches, commit history, and configuration are copied.
- ▶ The local repository is automatically connected to the remote repository (named `origin`).

Example: If you want to clone the Bootstrap repository, you would run.

- ▶ `git clone https://github.com/twbs/bootstrap`
 - ▶ The Bootstrap repository needs to be cloned to the local system.
 - ▶ Copy the repository URL to use it for cloning.
 - ▶ Navigate to the target directory where the repository should be stored using the `cd` command.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~ (master)
$ cd Desktop/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop (master)
$ mkdir Git

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop (master)
$ cd Git/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ █
```

► After that, Clone the repository using `git clone <repo-link>`

► `git clone https://github.com/twbs/bootstrap`

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 181364, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (13/13), done.
Receiving objects: 14% (25606/181364), 16.08 MiB | 839.00 KiB/s
```

- The cloning process starts, displaying progress messages that indicate the percentage of the repository data being downloaded (e.g., 14%, 50%, 100%).

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 184546, done.
remote: Total 184546 (delta 0), reused 0 (delta 0), pack-reused 184546 eceiving objects: 100% (184546/184546), 208
Receiving objects: 100% (184546/184546), 209.26 MiB | 1.59 MiB/s, done.
Resolving deltas: 100% (120314/120314), done.
Updating files: 100% (712/712), done.
```

- **NOTE**- If you need to stop the cloning process before it finishes, you can press Ctrl + C to cancel the operation.

- Once the cloning process is complete, navigate into the new directory (e.g., `cd bootstrap`) to start working on the project locally.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ ls
bootstrap/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ cd bootstrap/

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git/bootstrap (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git/bootstrap (main)
$ git log
commit c3c65911665ab64bdaa15d405db65ee81655dbf3 (HEAD -> main, origin/main, origin/HEAD)
Author: Kitty Giraudel <1889710+KittyGiraudel@users.noreply.github.com>
commit c3c65911665ab64bdaa15d405db65ee81655dbf3 (HEAD -> main, origin/main, origin/HEAD)
Author: Kitty Giraudel <1889710+KittyGiraudel@users.noreply.github.com>
Date: Tue Aug 9 16:14:51 2022 +0200
```

Update an outdated name mentioned in a Sass comment

Git Clone --branch

The `git clone --branch` command is used to clone a specific branch from a remote repository instead of the default branch.

- ▶ Allows cloning a specific branch from a remote repository.
- ▶ By default, `git clone` clones the main (or master) branch.
- ▶ Useful when you only need to work with a particular branch.
- ▶ Syntax:
 - ▶ `git clone --branch <branch_name> <remote_repository_url>`
- ▶ `<branch_name>`: This is the name of the branch you want to clone from the remote repository.
- ▶ `<repository_url>`: This is the URL of the remote repository (either HTTPS or SSH).
- ▶ For example, if you want to clone the `feature-xyz` branch from a repository, you would run the following command:
 - ▶ `git clone --branch feature-xyz https://github.com/username/repository`

Git supports two specialized cloning methods: mirror cloning and bare cloning.

1. **Git Clone - mirror:**

- ▶ A mirror clone creates an **exact copy of the remote repository**, including all branches, tags, and references.
- ▶ Use Case: Commonly used for repository backups or migration.
- ▶ `git clone --mirror <repository-url>`

2. **Git Clone - bare:**

- ▶ A bare clone contains **only the repository metadata** and version control data without a working directory.
- ▶ Use Case: Typically used for central repositories on servers for collaboration.

Check-in and Commit of Git Repositories

- ▶ In Git, **commit** is the official operation used to save changes to the local repository.
- ▶ The term **check-in** is commonly used in older version control systems such as SVN (Subversion), but in Git it generally refers to the process of **staging changes and committing them**.
- ▶ A commit creates a permanent snapshot of the project at a specific point in time.

Difference Between Check-in and Commit

Check-in	Commit
General term for saving changes to a repository.	Specific Git command that records changes in the local repository.
In Git, it involves git add followed by git commit.	Performed using the git commit command.
May refer to uploading changes in some version control systems.	Saves changes only in the local repository until pushed.
Not an official Git command.	Official Git command.

- It saves the state of your project, including all tracked files.
- When you make a commit in Git, it is simply added to the currently active branch.



- Each commit has a **unique identifier (SHA-1 hash)**, allowing you to track changes accurately.
- A commit usually includes a **commit message**, describing the purpose of the change. For example :
`git commit -m "message"`
- Commits help in **version control**, enabling rollback to previous states if needed.
- You can see the history of your commits using `git log` to understand how your project has changed over time.

How to use git commit ?

- ▶ Stage your changes by any one below command:
 - ▶ `git add <file-name>` # Stages a specific file
 - ▶ `git add .` # Stages all changes in the directory
- ▶ Commit your changes:
 - ▶ `git commit -m "Your commit message here"` # Commits with a message
- ▶ The command will show the following:

MINGW64:/c/Users/Deepak Panwar/Desktop/gitdemo

added all the files in staging area ← `Deepak Panwar@DESKTOP-HV7J4HS MINGW64 ~/Desktop/gitdemo (master)`
`$ git add .`

git commit command is used
to save the changes from
the staging area to the Git
repository. ← `Deepak Panwar@DESKTOP-HV7J4HS MINGW64 ~/Desktop/gitdemo (master)`
`$ git commit -m "first commit"`
[master (root-commit) 7dec382] first commit
2 files changed, 2 insertions(+)
create mode 100644 aaa.txt
create mode 100644 bbb.txt

`Deepak Panwar@DESKTOP-HV7J4HS MINGW64 ~/Desktop/gitdemo (master)`
`$`

Different ways to use git commit

1. Commit with a Message:

- ▶ **git commit -m** "Your commit message"
- ▶ Saves staged changes with a short description.
- ▶ This command is the most commonly used way to commit changes in Git.

2. Commit Only Specific Files:

- ▶ **git commit filename.txt -m** "Commit message"
- ▶ Commits only the specified file(s).

3. Commit with a Default Message Editor:

- ▶ **git commit**
- ▶ Opens the default editor to write a commit message.

4. Commit All Tracked Files (Skip git add):

- ▶ **git commit -a -m** "Your commit message"
- ▶ Automatically stages and commits modified and deleted files (but not new untracked files).

5. Amend the Last Commit:

- ▶ **git commit --amend -m** "Updated commit message"
- ▶ Modifies the last commit (useful for fixing commit messages or adding missed changes).

Fetching the Repositories

- ▶ **Git Fetch** (`git fetch`) is a command that retrieves the latest commits, files (branches/tags) from a remote repository into your local repository **without merging** them into your active working directory.
- ▶ It allows developers to see what teammates have pushed to the remote server before integrating those changes into their local code.
- ▶ **Key Characteristics**
 - **Non-Destructive / Safe:** Because it does not modify your working directory or current branch, it is completely safe to execute at any time. It never overwrites local work.
 - **Tracking Remote Branches:** Changes are downloaded into remote-tracking branches (e.g., `origin/main` or `origin/dev`).

- git fetch is a safe and non-disruptive Git command that checks for and downloads the latest updates from a remote repository without merging them into your local branch, giving developers full control over when and how changes are integrated.
- Prevents automatic merges that may occur with git pull.
- Retrieves remote branches, tags, and references without altering local files.
- Allows developers to review and compare changes before merging them manually.

•**Analogy:**

Git Fetch is like **checking the latest books available in a library without borrowing them**. You can see what's new but nothing changes in your own collection.

Working of Git Fetch

- ▶ git fetch works by safely retrieving the latest updates from a remote repository without altering the local working branch.
- ▶ Git connects to the remote repository (commonly origin).
- ▶ It checks for new commits, branches, or updates on the remote.
- ▶ The updates are downloaded and stored as remote-tracking branches (for example, origin/main).
- ▶ Local branches remain unchanged during this process.
- ▶ Developers can later merge or rebase these fetched changes as needed.

When to Use Git Fetch

Here are some cases where git fetch is particularly useful:

- ▶ **Collaborative Projects:** Fetch updates to stay informed about team changes.
- ▶ **Monitoring Remote Branches:** Track remote progress without modifying local branches.
- ▶ **Inspecting Changes:** Review updates before merging or rebasing.

Syntax

`git fetch <remote> <branch>`

- <remote>**: The name of the remote repository (default is origin).
- <branch> (optional)**: The specific branch you want to fetch.

Example:

A] `git fetch origin`

Fetches updates for all branches from the remote repository named origin.

B] `git fetch origin feature-branch`

Fetches only the updates from the feature-branch on the remote repository.

C] `git fetch origin dev`

Fetches the latest changes from the remote dev branch for review.

D] `git log origin/dev`

Displays the commit history of the fetched remote dev branch.

E] `git fetch --all --tags`

Fetches all branches and tags from all configured remote repositories.

Git Fetch

Git Pull

Checks for updates from remote without applying them

Fetches and merges changes into your current branch

Safe for reviewing before merging

May introduce merge conflicts immediately

Keeps local working directory unchanged

Alters working directory based on remote updates

Best for cautious syncing

Best when you're ready to integrate updates

Advanced Usage of Git Fetch

1. Fetching and Pruning Deleted Branches

Over time, remote branches may be deleted. You can clean up stale branches with:
`git fetch --prune`

This command removes references to branches that no longer exist on the remote repository.

2. Fetching Specific Tags

If you want to fetch a particular tag, use:
`git fetch origin tag v1.0.0`

3. Shallow Fetching

In cases where bandwidth or storage is limited, you can perform a shallow fetch:
`git fetch --depth=1`

This fetches only the latest commit, reducing the amount of data downloaded.

► **Best Practices**

- **Frequent Fetching:** Regularly use `git fetch` to keep your local repository aware of the latest changes in the remote.
- **Inspect Before Merging:** Always inspect fetched changes using `git log` or `git diff` before merging them into your working branch.
- **Prune Stale Branches:** Use `git fetch --prune` to keep your branch list clean and up-to-date.

Git Pull

Git pull is a command which is used to fetch and integrate the changes which are present in the remote repository to the local repository.

- **Git pull is a combination of git fetch and git merge.**
- It is used to update the local branch with changes from the remote repository branch.
- Ensures that all developers have the latest code on their local machines.
- Allows developers to work with newly updated code without conflicts.

Syntax

```
git pull [remote-name] [branch-name]
```



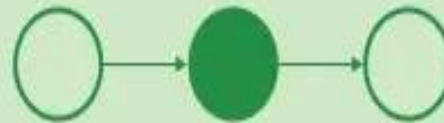
Pull Request



Notifications



Discussion



Follow-Up Commits

The diagram explains the **Pull Request (PR) process in Git/GitHub**. A pull request is a request made by a developer to **merge their code changes into another branch**, usually the main branch.

Step-by-step explanation

1. Pull Request

A developer completes some changes in their own branch.

They create a **Pull Request** asking the team to review and merge those changes.

The PR contains information about **what was changed and why**.

2. Notifications

Team members or reviewers receive **notifications** about the new pull request.

They can be informed that their review or approval is required.

3. Discussion

Reviewers examine the code and can **comment or discuss** the changes.

They may suggest improvements, ask questions, or identify errors.

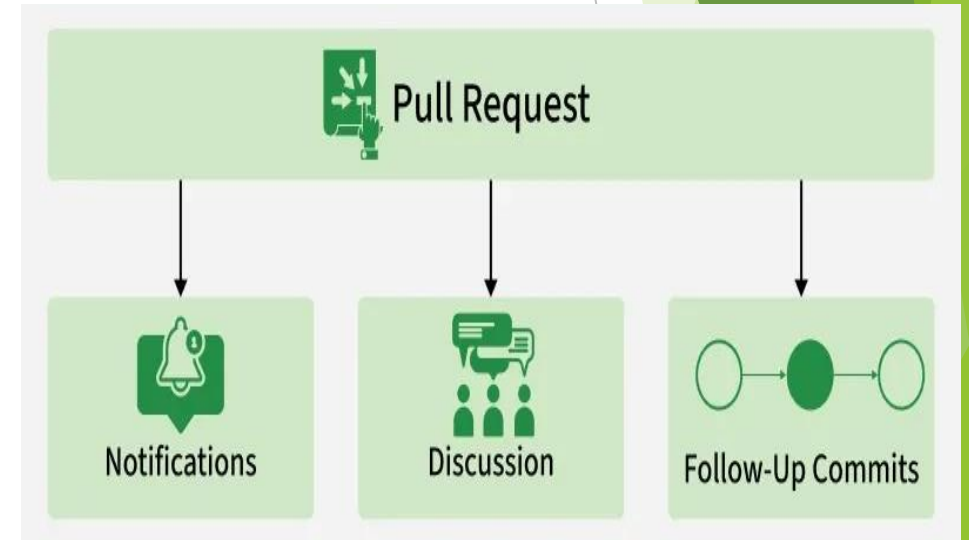
The developer can respond and make the required changes.

4. Follow-Up Commits

If changes are requested, the developer makes additional changes.

These changes are committed and pushed to the same branch.

The Pull Request is automatically updated with these new commits



Workflow of Git Pull

- ▶ Illustrates the main Pull Request workflow.
- ▶ Shows notifications sent to team members.
- ▶ Highlights discussion and code review.
- ▶ Indicates follow-up commits before merging.
- ▶ Explains iterative collaboration prior to final merge.

Git Pull Examples

Example 1: To pull from the default remote and current branch

```
git pull
```

Example 2: Pulling From a Specific Remote and Branch

```
git pull <remote-name> <branch-name>
```

Example 3: Pulling and Rebasing.

```
git pull --rebase
```

When working with Git, **git pull** and **git pull --rebase** are two different ways to integrate remote changes from a target branch into your local branch

Standard Pull (git pull)

- ▶ By default, git pull executes a git fetch followed by a git merge.
- ▶ **How it works:** It downloads the commits from your remote repository and creates a new merge commit that ties together the history of the remote branch and your local branch.
- ▶ **Commit History:** Creates a non-linear, branching history with explicit merge commits whenever local and remote histories diverge.
- ▶ **Best for:** Shared or public branches where preserving the exact chronological sequence of when branches were created and merged is required.

Pull with Rebase (git pull --rebase)

- ▶ Running git pull --rebase executes a git fetch followed by a git rebase.
- ▶ **How it works:** It temporarily stashes/detaches your local, un-pushed commits. It updates your local branch to match the latest commit on the remote branch. It replays your local commits one-by-one on top of the newly updated remote branch history.
- ▶ **Commit History:** Produces a clean, linear, single-line history without unnecessary merge commits.
- ▶ **Best for:** Feature branches and individual developer workflows before merging into main, keeping history tidy and easier to trace using commands like git log or git bisect.

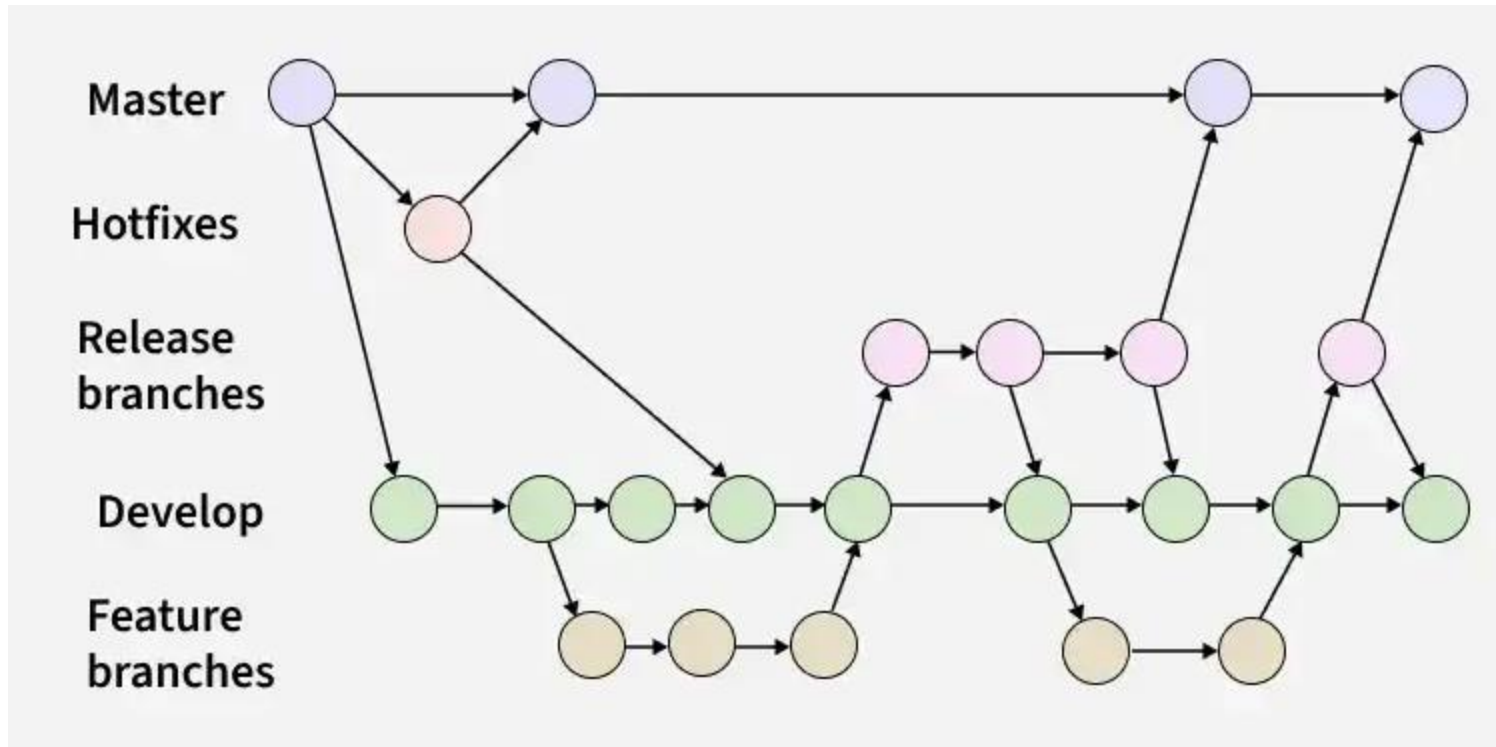
Git Branching Technique

- ▶ A branching techniques defines how developers create, manage and merge branches in a version control system like Git to ensure smooth collaboration and organized code development.
- ▶ Provides clear rules for writing, merging and deploying code.
- ▶ Helps keep the repository structured and maintainable.
- ▶ Reduces merge conflicts when multiple developers work simultaneously.
- ▶ **Techniques:**
 - ▶ **Feature Branching**
 - ▶ **GitFlow**
 - ▶ **GitHub Flow**
 - ▶ **GitLab Flow**
 - ▶ **Trunk-Based**

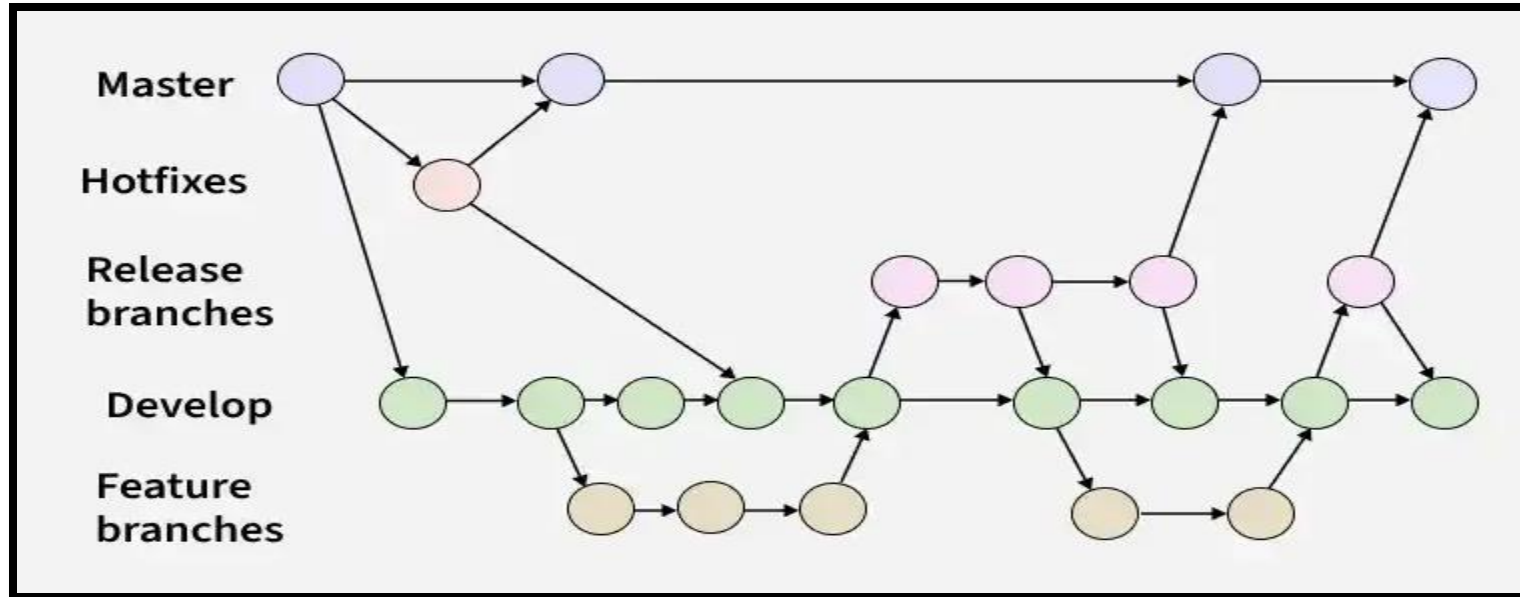
Common Git Branching Strategies

1. GitFlow Workflow

- GitFlow enables parallel development, allowing developers to work separately on feature branches. A feature branch is created from a master branch and after completion of changes the feature branch is merged with the master branch.



- **Master:** Used for product release
- **Develop:** Used for ongoing development
- **Feature Branches:** Created from the develop branch to work on specific features.
- **Release Branches:** Created from the develop branch to prepare for production releases and bug fixes
- **Hotfix Branches:** Created from the master branch to address urgent issues directly in production. It helps in addressing discovered bugs smoothly, allowing developers to continue their work on the develop branch while the issue is resolved.



2. GitHub Flow

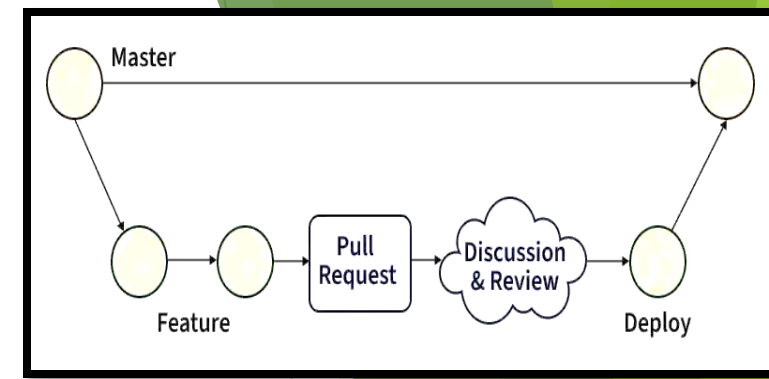
GitHub Flow is a lightweight branching strategy that keeps the main branch always deployable and supports fast, continuous development.

- ▶ Uses only short-lived feature branches created from and merged back into the main branch.
- ▶ No separate release branches, making the workflow simple and easy to manage.
- ▶ Ideal for fast releases with continuous integration and delivery.

The types of branches that are present in GitHub Flow are:

- ▶ **Master:** The GitHub Flow starts with the master branch, which contains the most recent stable code ready for release.
- ▶ **Feature:** Developers create feature branches from the main branch to work on features or fixes, then merge them back after completion, resolving any conflicts before finalizing the merge.

GitHub Flow



Step

What happens

Easy explanation

1. Master/Main

Start from the Master/Main branch

This contains the **stable version** of the project.

2. Feature Branch

Create a feature branch from Master

A developer creates a **separate branch** to work on a new feature without disturbing the main code.

3. Develop & Commit

Work is done on the feature branch

The developer writes code, tests it, and makes commits.

4. Pull Request

A Pull Request is created

The developer **requests the team to check and merge** the changes into Master/Main.

5. Discussion & Review

Team reviews the code

Team members discuss the changes, find problems, and suggest improvements.

6. Merge

Approved changes are merged

After successful review, the feature is added to the main branch.

7. Deploy

Application is deployed

The updated code is released so that users can use the new feature.

8. Back to Master

New version reaches Master

Master/Main now contains the latest approved and deployed changes.

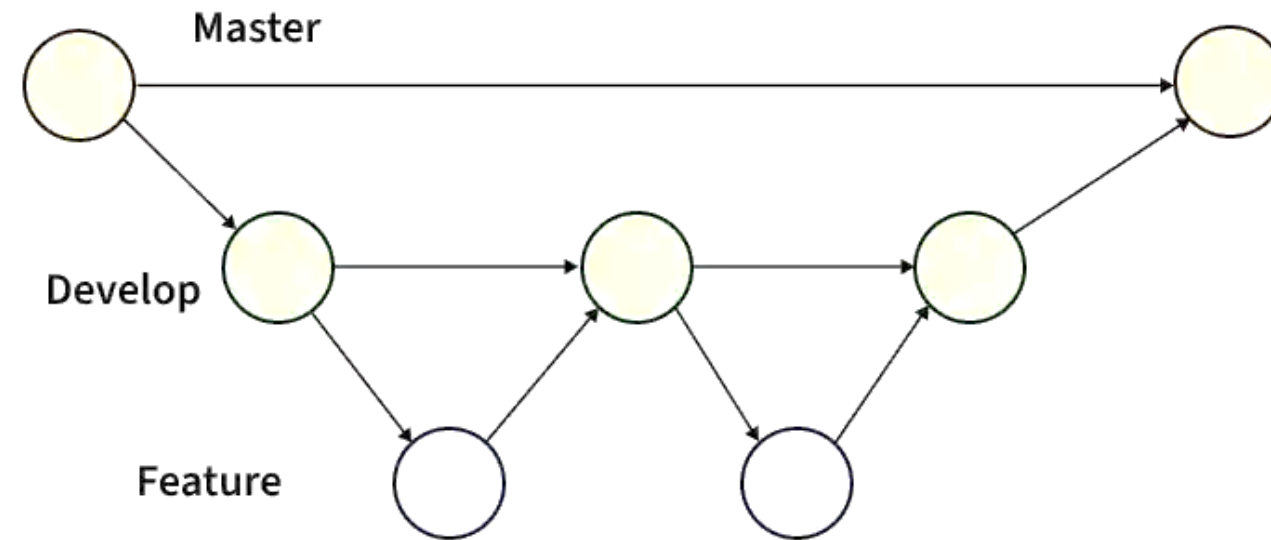
3. GitLab Flow

GitLab flow is a scalable branching strategy that combines flexibility with continuous integration while keeping the master branch stable.

- ▶ Designed for teams using GitLab with built-in support for CI/CD and automated testing.
- ▶ Provides a flexible workflow that balances structured releases with continuous development.

The types of branches that can be present in GitFlow are:

- **Master:** Main production branch housing stable release ready code.
- **Develop:** Contains new features and bug fixes.
- **Feature:** Feature branches are created from the develop branch to work on new features or fixes, then merged back into develop.
- **Release:** A release branch is created from develop to prepare a new release and later merged into both develop and main branches.



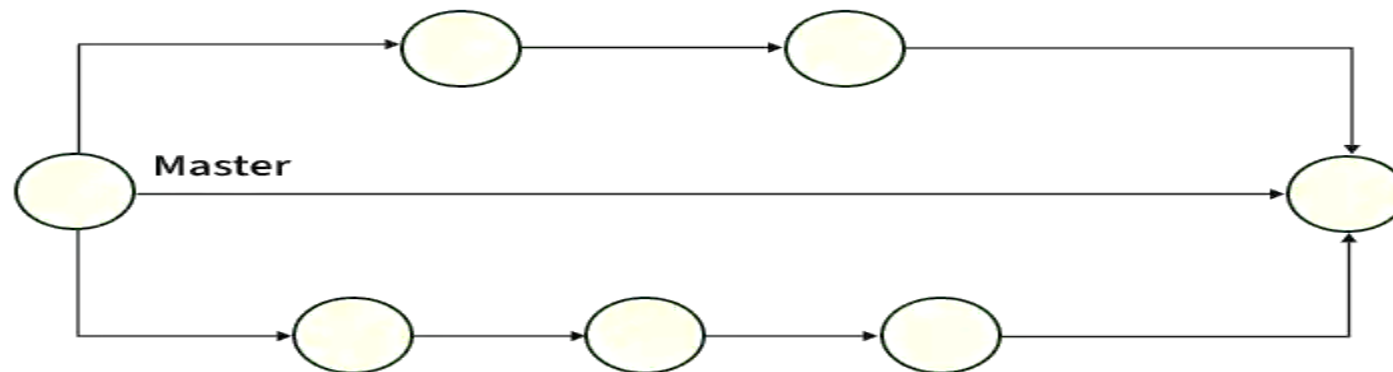
Feature Behind Feature Flag 2

4. Trunk Based Development

Trunk-Based Development is a branching strategy where all developers work directly on a single main branch, keeping it always in a release-ready state.

- ▶ **Development happens on one primary branch** (usually main or master) without long-lived branches.
- ▶ Feature flags are used to hide incomplete features until they are ready.
- ▶ Encourages small, frequent commits to reduce merge conflicts.
- ▶ **Supports continuous integration and continuous delivery (CI/CD).**
- ▶ Best suited for **small teams or projects** that prefer a simple and fast workflow.

Feature Behind Feature Flag 1



Feature Behind Feature Flag 2

Master



Two different features are developed:

Feature 1 → Feature 1 code changes → Feature 1 completed

Feature 2 → Feature 2 code changes → Feature 2 completed

Both eventually come back to:

Master

The important point is that **Feature 1 and Feature 2 can be present in the main code but hidden from users using feature flags.**

- ▶ **Feature Flags allow developers to put unfinished features into the main branch without immediately showing them to users.**
- ▶ This makes **Trunk-Based Development** safer because developers can make **small, frequent changes** while keeping the main branch stable.

Comparison of Git Branching Techniques

Technique	Simple Idea	Best For	Main Advantage
Feature Branching	One branch for each feature	Small/medium projects	Easy feature isolation
GitFlow	Many branches for development, release and fixes	Large projects	Organized release management
GitHub Flow	Feature → Pull Request → Main	Web applications	Simple and fast
GitLab Flow	Feature → Development → Release → Production	CI/CD projects	Good environment/release control
Trunk-Based	Mostly one main branch	Agile/DevOps	Fast integration and deployment

Git branch command

A **Git branch** is an independent line of development that contains its own commits. It allows developers to work on new features or fixes without modifying the main project.

► Why to Use git branch:-

- **Isolated Line of Development:** A branch in Git represents an isolated line of development, enabling you to work on different features or bug fixes without affecting the main project.
- **Lightweight and Efficient:** Branching in Git is lightweight and efficient, making it ideal for experimenting with new ideas, implementing features, or resolving issues.
- **Keep the main branch Stable:** It helps in keeping the main branch stable while allowing parallel development in separate branches.
- **Collaboration:** In collaborative environments, it allows team members to work on different tasks independently and merge their work back into the main codebase when ready.

Different ways to use git branch

1.Create a New Branch:

- Syntax :
 - `git branch <branch-name>`
- Example :
 - `git branch feature-1`
- Use case:** Create a new branch from the current branch without switching to it.

2.List All Local Branches:

- Syntax & Example :
 - `git branch`
- Description:** Displays a list of all local branches in the repository.
- Use case:** To see which branches exist and which one you're currently on (highlighted with *).

3.Create a New Branch from a Specific Commit:

- Syntax :
 - `git branch <branch-name> <commit-id>`
- Example:
 - `git branch debug-fix a1b2c3d`
- Use case:** Useful when you want to branch out from a previous point in history.

4.Rename a Branch:

- Syntax :
 - `git branch -m <new-name>`
- Example :
 - `git branch -m dev`
- Use case:** Renames the current branch to a new name.

5. Delete a Local Branch:

- Syntax :
 - `git branch -d <branch-name>`
 - (use `-D` to force delete)
- Example:
 - `git branch -d old-feature`
- Use case:** Clean up branches that have already been merged or are no longer needed.

6.List Remote Branches:

- Syntax :
 - `git branch -r`
- Description:** Shows branches from remote repositories (e.g. `origin/main`, `origin/dev`).

3.List All Branches (Local + Remote):

- Syntax :
 - `git branch -a`
- Description:** Lists all local and remote branches.

4.Check Last Commit on Each Branch:

- Syntax :
 - `git branch -v`
- Description:** Shows the latest commit on each branch.

5.See Which Branches Have Been Merged:

- Syntax :
 - `git branch --merged`
- Description:** Lists branches that have already been merged into the

Basic Definitions

Term	Definition
Version Control System (VCS)	A tool used to track, manage and record changes to source code and maintain project history.
Centralized VCS (CVCS)	A version control system in which project files and version history are stored on a central server.
Distributed VCS (DVCS)	A version control system in which every developer has a local repository containing the project's history.
Git	An open-source distributed version control system developed by Linus Torvalds in 2005.
Repository	A Git-managed location containing project files and their complete version history.
Staging Area	An intermediate area where selected changes are prepared before they are committed.
Commit	A permanent snapshot of project changes stored in the local repository.
Clone	The process of creating a complete local copy of an existing remote Git repository.
Branch	An independent line of development used to work on features or fixes without directly affecting the main code.
HEAD	A pointer that indicates the latest commit in the current branch.

Thank you

